RESOLUÇÃO DOS 50 EXERCÍCIOS CONCEITUAIS E PRÁTICOS

ALUNO: KAUÃ FERNANDEZ FREITAS | 6º SEMESTRE DE CIÊNCIA DA COMPUTAÇÃO

Parte I — Arquitetura de sistemas (1–10)

01. [Conceitual] Defina arquitetura de software e explique sua importância para manutenção, desempenho e evolução de um sistema.

= Arquitetura de software é a forma como um sistema é estruturado para operar e evoluir. Ela define quais componentes existem, como se conectam, de que maneira dados e transações percorrem a solução e quais decisões garantem consistência, escala e segurança ao longo do tempo. Do modo de vista da manutenção, ela é a espinha dorsal, essencial ao isolar responsabilidades e reduzir o tempo de correção de falhas, crucial para o desempenho ao definir estratégias eficientes de escalabilidade, uso de recursos e gerenciamento de carga, e indispensável para a evolução por permitir a inserção de novas tecnologias e funcionalidades sem comprometer o funcionamento das estruturas.

02. [Conceitual] Diferencie arquitetura monolítica, cliente-servidor e microsserviços. Apresente uma vantagem e uma limitação de cada abordagem.

= Arquitetura Monolítica: Toda a aplicação (interface do usuário, regras de negócio e acesso ao banco de dados) é construída e executada como uma única unidade e base de código acoplada.

Por estar tudo em único pacote, é mais fácil de codificar, testar localmente e realizar o deploy inicial com baixo custo operacional; Já por outro lado, há dificuldade de escalabilidade e manutenção.

Arquitetura Cliente-Servidor: Divide o sistema em dois papéis distintos – o cliente (que solicita dados ou exibe a interface) e o servidor (que processa a requisição, gerencia os recursos e centraliza os dados).

O servidor concentra as regras críticas e o banco de dados, facilitando o controle de acesso, a integridade da informação e backups padronizados. Se o servidor inicial cair ou receber um volume excessivo de requisições simultâneas dos clientes, todo o sistema fica inoperante ou apresenta alta latência.

Arquitetura de Microsserviços: Divide a aplicação em múltiplos serviços pequenos, autônomos e especializados, onde cada um executa uma função de negócio própria e se comunica via APIs.

É possível escalar de forma independente apenas o serviço que está sob alta demanda (ex: módulo de pagamento), e a queda de um serviço que não compromete os demais.

Exige infraestrutura avançada para gerenciar sistemas distribuídos, lidar com comunicação via rede (latência, APIs), monitoramento (observabilidade) e orquestração de containers.

03. [Conceitual] Explique as responsabilidades das camadas de apresentação, negócio e persistência em uma arquitetura de três camadas.

= A apresentação recebe entradas e exibe resultados; a camada de negócio concentra regras, validações e decisões; a persistência acessa e mantém os dados. A separação reduz dependências e permite trocar, por exemplo, a interface sem reescrever as regras.

04. [Conceitual] Compare escalabilidade vertical e horizontal e indique um cenário adequado para cada uma.

= Escalabilidade vertical aumenta CPU, memória ou armazenamento de um servidor; é simples e adequada a um banco pequeno em crescimento. A horizontal adiciona instâncias e distribui carga; é adequada a aplicações web com muitos acessos e necessidade de alta disponibilidade.

05. [Conceitual] Explique os conceitos de acoplamento e coesão. Por que baixo acoplamento e alta coesão são desejáveis?

= Acoplamento é o grau de dependência entre módulos; coesão é o quanto as responsabilidades de um módulo pertencem ao mesmo

objetivo. Baixo acoplamento e alta coesão facilitam testes, manutenção, substituição de componentes e evolução do sistema.

06. [Prático] Desenhe a arquitetura de três camadas de um sistema de biblioteca: interface, regras de empréstimo e banco de dados. Identifique as trocas de informação.

= A interface envia identificador do leitor e do livro; a camada de negócio valida cadastro, disponibilidade e limites, registra o empréstimo e devolve uma confirmação; a persistência executa consultas e transações no banco.

Arquitetura de biblioteca

Interface (web/app) → Serviço de empréstimos

Serviço: verifica disponibilidade, limite e devolução

Repositório → Banco de dados (Livro, Leitor, Empréstimo)

07. [Prático] Para um aplicativo de delivery, identifique ao menos cinco componentes e descreva suas responsabilidades.

= O cliente exibe catálogo e pedidos; a API recebe requisições; a autenticação controla identidade e permissões; o serviço de pedidos calcula itens e status; o pagamento autoriza transações; notificações informam mudanças; catálogo e entrega mantêm produtos e acompanhamento.

Componentes do delivery

Cliente → API Gateway → Autenticação

Pedidos → Pagamentos → Notificações

Catálogo/Entrega: dados e acompanhamento do pedido

08. [Prático] Um sistema de matrícula apresenta lentidão em períodos de pico. Proponha duas mudanças arquiteturais e justifique como melhorariam a capacidade de atendimento.

= Primeiro, adicionar cache para consultas de cursos e distribuir requisições entre instâncias por um balanceador. Segundo, separar operações de escrita e tarefas demoradas em filas, usando réplicas de

leitura no banco. As medidas reduzem contenção e permitem escalar os pontos críticos.

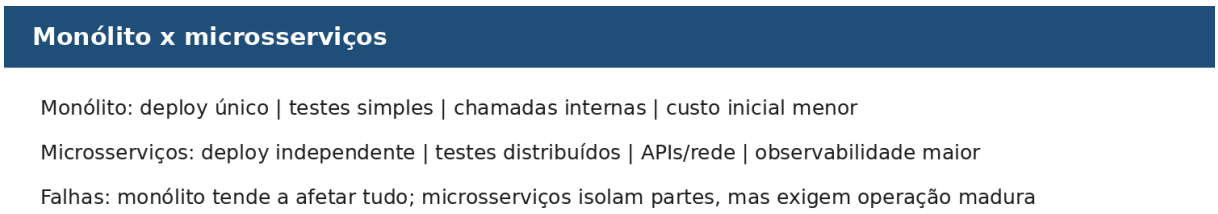
09. [Prático] Desenhe um fluxo de requisição HTTP, desde o navegador até o banco de dados, para consultar o histórico escolar de um aluno.

= O navegador envia uma requisição HTTPS; a API autentica e valida o identificador; o serviço consulta o repositório; o banco retorna disciplinas e notas; a API serializa JSON e o navegador renderiza a resposta.



10. [Prático] Crie uma tabela comparando monólito e microsserviços segundo implantação, testes, comunicação, custos operacionais e tolerância a falhas.

= A comparação visual resume os critérios solicitados: o monólito simplifica implantação e operação inicial, enquanto microsserviços permitem deploy e escala independentes, com maior complexidade de comunicação e observabilidade.



11. [Conceitual] Descreva as etapas de levantamento de requisitos, análise, projeto, implementação, testes, implantação e manutenção.

= Levantam-se necessidades e restrições; analisam-se viabilidade e prioridades; projeta-se a solução; implementa-se o código; testam-se comportamentos; implanta-se em ambiente real; e mantém-se o sistema com correções, segurança e evolução.

12. [Conceitual] Compare os modelos cascata, incremental e espiral em relação a mudanças de requisitos e gestão de riscos.

= Cascata segue fases sequenciais e lida pior com mudanças tardias. Incremental entrega partes funcionais e recebe feedback ao longo do projeto. Espiral repete ciclos guiados por riscos, sendo útil quando há incerteza técnica ou de negócio.

13. [Conceitual] Explique a diferença entre requisito funcional e não funcional. Dê três exemplos de cada para um sistema hospitalar.

= Funcionais descrevem serviços: cadastrar paciente, agendar consulta e registrar prescrição. Não funcionais definem qualidades ou restrições: resposta em até dois segundos, disponibilidade de 99,9% e controle de acesso por perfil.

14. [Conceitual] Diferencie Scrum, Kanban e práticas de DevOps, indicando o foco principal de cada abordagem.

= Scrum organiza trabalho em iterações, papéis e eventos. Kanban visualiza fluxo e limita trabalho em andamento. DevOps aproxima desenvolvimento e operações com automação, integração contínua, entrega contínua e monitoramento.

15. [Conceitual] Explique a diferença entre testes unitários, de integração, de sistema e de aceitação.

= Unitários isolam funções; integração verifica componentes juntos; sistema avalia o produto completo; aceitação confirma com critérios do negócio se a solução atende ao usuário.

16. [Prático] Escreva cinco histórias de usuário no formato 'Como [perfil], quero [função], para [benefício]' para um sistema acadêmico.

= 1. Como aluno, quero consultar minhas disciplinas, para planejar meu semestre. 2. Como aluno, quero emitir histórico, para comprovar minha formação. 3. Como professor, quero lançar notas, para manter o desempenho atualizado. 4. Como secretaria, quero abrir turmas, para organizar a oferta. 5. Como coordenador, quero acompanhar vagas, para evitar conflitos de matrícula.

17. [Prático] Monte um backlog com oito tarefas para desenvolver um cadastro de produtos. Classifique cada tarefa por prioridade e estimativa de esforço.

= Backlog: 1) modelo de dados — Alta/3 pontos; 2) tela de cadastro — Alta/5; 3) validação de campos — Alta/3; 4) API CRUD — Alta/5; 5) busca e filtros — Média/5; 6) edição e exclusão — Média/3; 7) controle de acesso — Alta/5; 8) testes e documentação — Média/3.

18. [Prático] Planeje uma sprint de duas semanas: defina objetivo, entregas, critérios de aceite e riscos para uma tela de login.

= Objetivo: permitir autenticação segura. Entregas: tela, endpoint, validações, mensagens e testes. Aceite: credenciais válidas entram; inválidas recebem mensagem; senha não é exposta; campos obrigatórios são validados. Riscos: integração com identidade, indisponibilidade do serviço e requisitos de segurança incompletos.

19. [Prático] Escreva cinco casos de teste para a função 'recuperar senha', incluindo pré-condições, entradas e resultados esperados.

= T1 usuário existente e e-mail válido → token enviado. T2 e-mail inexistente → resposta neutra, sem revelar cadastro. T3 formato inválido → validação no formulário. T4 token expirado → redefinição recusada. T5 nova senha curta ou igual à anterior → regra recusada com mensagem adequada.

20. [Prático] Crie um quadro Kanban com as colunas 'A fazer', 'Em andamento', 'Em teste' e 'Concluído'. Distribua dez tarefas de um projeto fictício.

= As dez tarefas foram distribuídas no quadro visual: três a fazer, duas em andamento, duas em teste e três concluídas.

Quadro Kanban
A fazer: autenticação, tela de perfil, relatório
Em andamento: API de pedidos, catálogo
Em teste: cadastro de usuário, checkout
Concluído: repositório, ambiente, modelo de dados

21. [Conceitual] Diferencie linguagens compiladas e interpretadas, observando que muitas plataformas combinam compilação e execução por máquina virtual.

= Compiladas são traduzidas antes da execução, geralmente gerando código nativo. Interpretadas são avaliadas por um runtime durante a

execução. Há modelos híbridos: Java compila para bytecode e a JVM interpreta ou compila esse bytecode com JIT.

22. [Conceitual] Explique programação procedural, orientada a objetos e funcional, citando uma característica marcante de cada paradigma.

= Procedural organiza instruções e procedimentos; orientação a objetos modela estado e comportamento em objetos; funcional privilegia funções, composição e imutabilidade, reduzindo efeitos colaterais.

23. [Conceitual] Compare tipagem estática e dinâmica, com exemplos de linguagens que utilizam cada abordagem.

= Na tipagem estática, tipos são verificados antes da execução, como em Java e C#. Na dinâmica, os tipos são associados em tempo de execução, como em Python e JavaScript. Ambas podem ser seguras quando usadas com boas validações e testes.

24. [Conceitual] Descreva os papéis de HTML, CSS, JavaScript e de uma linguagem de back-end em uma aplicação web.

= HTML estrutura o conteúdo; CSS define apresentação e responsividade; JavaScript implementa interação no navegador; o back-end processa regras, autenticação, persistência e integrações no servidor.

25. [Conceitual] Explique a diferença entre variável, constante, função, classe e objeto.

= Variável referencia um valor que pode mudar; constante representa um valor que não deve ser reatribuído; função encapsula comportamento; classe é um molde; objeto é uma instância concreta da classe.

26. [Prático] Escreva em Python um programa que leia três notas, calcule a média e informe se o aluno foi aprovado (média igual ou superior a 7).

= Código:
```
notas = [float(input(f"Nota {i+1}: ")) for i in range(3)]
media = sum(notas) / 3
print(f"Média: {media:.2f}")
print("Aprovado" if media >= 7 else "Reprovado")
```

27. [Prático] Crie uma função em Python que receba uma lista de preços e retorne o total, aplicando desconto de 10% quando o valor ultrapassar R$ 500.

= Código: def total_com_desconto(precos): total = sum(precos) return total * 0.90 if total > 500 else total

28. [Prático] Implemente em JavaScript uma função que valide um formulário de cadastro, exigindo nome não vazio, e-mail com formato básico válido e senha de pelo menos oito caracteres.

= Código: function validar({nome, email, senha}) { return nome.trim() !== "" && /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email) && senha.length >= 8; }

29. [Prático] Escreva pseudocódigo para buscar um produto pelo código em uma lista. Informe o resultado quando encontrado e quando inexistente.

= Pseudocódigo: buscar(lista, codigo): para cada produto em lista, se produto.codigo = codigo, retornar produto; ao final, retornar "Produto inexistente".

30. [Prático] Escolha Python, Java ou JavaScript e modele uma classe Produto com código, nome, preço e método para calcular preço com desconto. Mostre um exemplo de uso.

= Em Python: class Produto: def __init__(self, codigo, nome, preco): self.codigo, self.nome, self.preco = codigo, nome, preco def com_desconto(self, percentual): return self.preco * (1 - percentual/100)
p = Produto("A1", "Teclado", 100); print(p.com_desconto(10))

31. [Conceitual] Defina SGBD e explique os conceitos de tabela, registro, campo, chave primária e chave estrangeira.

= SGBD é o software que cria, consulta, protege e administra bancos de dados. Tabela organiza registros; registro é uma linha; campo é uma coluna; chave primária identifica unicamente uma linha; chave estrangeira referencia a chave de outra tabela.

32. [Conceitual] Compare MySQL, Oracle Database e dBase quanto a contexto histórico, aplicações típicas, modelo de dados e recursos disponíveis. Evite afirmar que são equivalentes.

= MySQL é um SGBD relacional amplamente usado em aplicações web. Oracle Database é uma plataforma corporativa robusta, com recursos avançados de segurança, alta disponibilidade e administração. dBase é uma tecnologia histórica de arquivos/tabelas, importante no desenvolvimento de aplicações desktop, mas não equivalente às plataformas corporativas atuais.

33. [Conceitual] Explique integridade referencial e a importância das restrições NOT NULL, UNIQUE e FOREIGN KEY.

= Integridade referencial mantém relações válidas entre tabelas. NOT NULL impede ausência de valor obrigatório; UNIQUE evita duplicidade; FOREIGN KEY garante que uma referência exista na tabela relacionada.

34. [Conceitual] Descreva as propriedades ACID de transações e dê um exemplo de sua importância em operações financeiras.

= Atomicidade executa tudo ou nada; consistência preserva regras; isolamento evita interferência indevida; durabilidade mantém o resultado após confirmação. Em uma transferência, o débito e o crédito devem ser confirmados juntos, nunca apenas um deles.

35. [Conceitual] Explique normalização e diferencie, em linhas gerais, primeira, segunda e terceira formas normais.

= Normalização reduz redundância e anomalias. 1FN exige valores atômicos; 2FN remove dependências parciais de parte de uma chave composta; 3FN remove dependências transitivas entre atributos não-chave.

36. [Prático] Modele um banco de dados de biblioteca com as entidades Livro, Leitor e Emprestimo. Defina atributos, chaves e relacionamentos.

= Livro possui id, título, autor e disponibilidade; Leitor possui id, nome e contato; Empréstimo possui id, leitor_id, livro_id, retirada e devolução. As chaves estrangeiras ligam empréstimos a um leitor e a um livro.

Modelo de dados da biblioteca

LEITOR (id PK, nome, email)

LIVRO (id PK, titulo, autor, disponivel)

EMPRESTIMO (id PK, leitor_id FK, livro_id FK, retirada, devolucao)

Relacionamentos: Leitor 1:N Empréstimo; Livro 1:N Empréstimo

37. [Prático] Escreva comandos SQL compatíveis com MySQL para criar a tabela Aluno (id, nome, email, data_nascimento), definindo chave primária e unicidade do e-mail.

= SQL: CREATE TABLE Aluno (id INT AUTO_INCREMENT PRIMARY KEY, nome VARCHAR(120) NOT NULL, email VARCHAR(160) NOT NULL UNIQUE, data_nascimento DATE);

38. [Prático] Escreva comandos INSERT para cadastrar três alunos e um SELECT que liste os nomes em ordem alfabética.

= SQL: INSERT INTO Aluno (nome,email,data_nascimento) VALUES ("Ana","ana@exemplo.com","2000-01-10"),("Bruno","bruno@exemplo.com","1999-05-20"),("Carla","carla@exemplo.com","2001-08-15"); SELECT nome FROM Aluno ORDER BY nome ASC;

39. [Prático] Considere tabelas Cliente(id, nome) e Pedido(id, cliente_id, valor). Escreva um SELECT com JOIN que mostre nome do cliente e valor do pedido.

= SQL: SELECT c.nome, p.valor FROM Cliente c INNER JOIN Pedido p ON p.cliente_id = c.id;

40. [Prático] Uma aplicação permite que duas pessoas atualizem o mesmo estoque. Descreva o problema de concorrência e proponha uma estratégia transacional para evitar inconsistências.

= O problema é a condição de corrida: duas leituras podem usar o mesmo estoque antigo e perder uma atualização. Use transação com UPDATE estoque SET quantidade = quantidade - ? WHERE id = ? AND quantidade >= ?, verifique as linhas afetadas, confirme ou faça rollback; em cenários necessários, use bloqueio de linha.

41. [Conceitual] Defina ferramentas CASE e diferencie apoio às fases de análise, projeto, implementação e manutenção.

= CASE significa Computer-Aided Software Engineering. Na análise apoia requisitos e casos de uso; no projeto apoia diagramas e modelos; na implementação pode gerar esqueletos e documentação; na manutenção auxilia rastreabilidade, impacto e engenharia reversa.

42. [Conceitual] Explique a utilidade de diagramas de casos de uso, classes e sequência na modelagem de sistemas.

= Casos de uso mostram atores e objetivos; classes mostram estrutura, atributos, operações e relacionamentos; sequência mostra mensagens e ordem temporal de uma interação.

43. [Conceitual] Diferencie ferramentas de modelagem, geradores de código e ferramentas de engenharia reversa.

= Ferramentas de modelagem criam representações do sistema; geradores transformam modelos em código ou artefatos; engenharia reversa parte de código ou banco existentes para reconstruir modelos e documentação.

44. [Conceitual] Apresente três benefícios e duas limitações do uso de ferramentas CASE em equipes de desenvolvimento.

= Benefícios: padronização, documentação consistente e rastreabilidade. Limitações: custo e curva de aprendizagem, além do risco de depender da ferramenta ou gerar modelos desatualizados quando a equipe não os mantém.

45. [Conceitual] Explique como rastreabilidade de requisitos e documentação técnica podem reduzir retrabalho.

= A rastreabilidade liga requisito, implementação e teste, permitindo localizar impactos quando algo muda. Documentação atualizada reduz interpretações divergentes, acelera onboarding e evita repetir investigações já realizadas.

46. [Prático] Utilizando uma ferramenta de diagramas, crie um diagrama de casos de uso para um sistema de biblioteca com três atores e seis casos de uso.

= O diagrama apresenta Leitor, Bibliotecário e Administrador e seis casos de uso: pesquisar livro, reservar livro, consultar empréstimos, registrar empréstimo, registrar devolução e cadastrar livro.

Casos de uso da biblioteca

Atores: Leitor | Bibliotecário | Administrador

Leitor: pesquisar livro, reservar, consultar empréstimos

Bibliotecário: registrar empréstimo, registrar devolução

Administrador: cadastrar livro, gerenciar usuários

47. [Prático] Crie um diagrama de classes com Aluno, Curso e Matricula. Defina atributos, operações e multiplicidades dos relacionamentos.

= Aluno e Curso se relacionam por meio de Matricula. Um aluno pode ter várias matrículas e um curso pode possuir várias matrículas; cada matrícula pertence a exatamente um aluno e um curso.

Diagrama de classes

Aluno {id, nome; matricular()}

Curso {codigo, nome; ofertar()}

Matricula {data, status; cancelar()}

Aluno 1..* — Matricula — *..1 Curso

48. [Prático] Modele um diagrama de sequência do processo de login, envolvendo usuário, interface, serviço de autenticação e banco de dados.

= O usuário informa credenciais; a interface chama o serviço; o serviço consulta o banco e compara o hash; em sucesso retorna sessão/token e, em falha, uma mensagem genérica.

Sequência de login

Usuário → Interface: informa credenciais

Interface → Serviço de autenticação: autenticar()

Serviço → Banco: buscar usuário e verificar hash

Banco → Serviço → Interface: token ou erro

49. [Prático] Use uma ferramenta de modelagem de dados para desenhar um diagrama entidade-relacionamento de uma loja virtual com Cliente, Produto, Pedido e ItemPedido.

= Cliente possui pedidos; pedido possui itens; cada item referencia um produto e registra quantidade e preço no momento da compra. Isso preserva o histórico mesmo que o preço do produto mude.

DER da loja virtual

Cliente 1:N Pedido

Pedido 1:N ItemPedido

Produto 1:N ItemPedido

ItemPedido: quantidade, preço_unitário; FKs pedido_id e produto_id

50. [Projeto integrador] Proponha um sistema de agendamento de consultas. Entregue requisitos, arquitetura, linguagem, modelo de dados, diagramas, testes e plano de quatro semanas.

= Sistema proposto: requisitos funcionais: cadastrar paciente, cadastrar profissional, consultar horários, agendar consulta e cancelar/reagendar. Não funcionais: autenticação por perfil, resposta usual abaixo de dois segundos e auditoria das alterações. Arquitetura: apresentação web, API de negócio e persistência relacional. Linguagem: Python com FastAPI, por produtividade e ecossistema web; JavaScript no cliente. Entidades: Paciente, Profissional, Especialidade, Agenda, Consulta e Usuário. Diagramas: casos de uso e sequência do agendamento. Testes: cadastro válido, conflito de horário, cancelamento, acesso sem permissão e indisponibilidade do banco. Plano: semana 1 requisitos/modelagem; semana 2 API e banco; semana 3 interface e integração; semana 4 testes, correções e implantação piloto.